Single Trajectory Based Metaheuristics I

Heuristic Optimization Techniques 192.162, VU 6 ECTS

Enrico Iurlano, Günther Raidl, and Laurenz Tomandl
heuopt@ac.tuwien.ac.at

Algorithms and Complexity Group
Institute of Logic and Computation, TU Wien

WS 2026

Version from October 22, 2026



Integrating Local Search into (meta)heuristics

Simple extensions of Local Search: RRLS & ILS

- Random Restart Local Search (RRLS): Metaheuristic performing LS many times from different random solutions and taking the overall best solution.
- Iterated Local Search (ILS): Metaheuristic alternating between intensification (local search) and diversification (perturbation).
- ILS appeared in different forms since the 1980s; formalized by Lourenço et al. (2003).
  - Start from one feasible solution and apply a LS to reach a local optimum.
  - To escape that optimum, apply a controlled perturbation (e.g. swap, scramble, destroy-and-repair).
  - Use an acceptance criterion to decide if the perturbed solution after refinement by local search becomes the new incumbent.
  - Simple to implement and easy to hybridize with other strategies (Tabu, Simulated Annealing, population-based methods $\rightsquigarrow$ see later chapters of the lecture).

Simple extensions of Local Search: Iterated Local Search

procedure Iterated Local Search

begin
  $s_0 \leftarrow$ GenerateInitialSolution();
  $s^* \leftarrow$ LocalSearch(s_0);
  $s_{\text{best}} \leftarrow s^*$;
  repeat
    $s' \leftarrow$ Perturbation(s^*);
    $s'' \leftarrow$ LocalSearch(s');
    if AcceptanceCriterion(s^*,s'') **then**
      $s^* \leftarrow s''$;
    end if;
    if s'' is better than s_{best} **then**
      $s_{\text{best}} \leftarrow s''$;
    end if;
  until stopping criteria satisfied;
  return s_{best};
end

Greedy Randomized Adaptive Search Procedure (GRASP)

Greedy Randomized Adaptive Search Procedure (GRASP)

Basic idea:

- A greedy construction heuristic is systematically randomized to generate a diverse set of promising initial solutions for subsequent Local Search.
- First approaches from Feo and Resende (1989)

GRASP

procedure GRASP

begin
  $x^* \leftarrow$ undefined;
  repeat:
    $x \leftarrow$ Randomized Greedy Heuristic;
    $x' \leftarrow$ Local Search (x);
    if x^* undefined **or** x' is better than x^* **then**
      $x^* \leftarrow x'$;
    end if;
  until stopping criteria satisfied;
  return x^*;
end

Randomization of Greedy Construction Heuristics

Question: How can the nearest neighbor heuristic for the TSP be randomized?

- Start the algorithm from a random node.
- Do not always take the nearest node as the next one, but choose a neighbor at random out of
  - the k nearest nodes, or
  - all the nodes whose distance to the current one is not greater than a certain distance (threshold).

Similar possibilities also hold for other heuristics like insertion heuristics where we can

- randomize the order of insertion of the nodes.
- insert the next node not always at the best position but choose the position at random among
  - the k best positions, or
  - all positions such that the corresponding insertion costs are below a certain threshold.

Randomization of Greedy Construction Heuristics

procedure Randomized Greedy Heuristic

begin
  $x \leftarrow$ empty solution;
  while x is not a complete solution **do**
    build CL;
    build RCL from CL;
    $e \leftarrow$ random selection from RCL;
    $x \leftarrow x \oplus e$;
  end;
  return x;
end

CL: **C**andidate **L**ist = elements that can be included in the partial solution
RCL: **R**estricted **C**andidate **L**ist = subset of CL with promising elements

General approaches to determine *RCL*

- Take the k cheapest solution components suitable to extend current partial solution
- Let $c_{\min}$ be the minimum and $c_{\max}$ the maximum costs for the components in *CL*. Add $e \in CL$ to *RCL* if

$$c(e) \leq c_{\min} + \alpha \cdot (c_{\max} - c_{\min}), \quad \alpha \in [0, 1]$$

Note:
$\alpha = 0 \quad \hat{=} \quad$ greedy construction heuristic
$\alpha = 1 \quad \hat{=} \quad$ random solution construction

Subsequent Local Search

Typically best improvement step function is used.

Benefits if Local Search is started from **multiple** randomized greedy solutions:

- Generally, a lower variance and a higher solution quality than performing Local Search on purely random solutions
- Local search usually requires few(er) iterations.
  - Faster.
  - In the same time more initial solutions can be improved.
- Randomization and repetition lead to better results than performing the greedy heuristic and Local Search only once (but also requires more time).

Example: GRASP for (MAX-)SAT

The following is an approach in the fashion of Feo and Resende (1995).

- Greedy function: number of so far unsatisfied clauses that would be satisfied after instantiating a variable x_i with 1 or 0
  - Let Φ_i^+: set of additionally satisfied clauses, if $x_i = 1$
  - Let Φ_i^-: set of additionally satisfied clauses, if $x_i = 0$

- *RCL* restrictions:
  - Let $\Phi^* = \max_{i=1,\ldots,n}\{|\Phi_i^+|, |\Phi_i^-| \mid x_i \text{ not instantiated yet}\}$
  - $|\Phi_i^+| \geq \alpha \cdot \Phi^* \;\rightarrow\; (x_i = 1) \in \mathit{RCL}$
  - $|\Phi_i^-| \geq \alpha \cdot \Phi^* \;\rightarrow\; (x_i = 0) \in \mathit{RCL}$
  - E.g., $\alpha = 0.5$

Example: GRASP for (MAX-)SAT

Imagine the following formula f:

$$f(x) = (x_1 \vee x_2) \wedge (x_1 \vee \overline{x_3}) \wedge (\overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_1} \vee x_4)$$

Then the values $|\Phi_i^+|$ and $|\Phi_i^-|$ are as follows:

$\|\Phi_1^+\| = 2$	$\|\Phi_2^+\| = 1$	$\|\Phi_3^+\| = 0$	$\|\Phi_4^+\| = 1$
$\|\Phi_1^-\| = 1$	$\|\Phi_2^-\| = 1$	$\|\Phi_3^-\| = 2$	$\|\Phi_4^-\| = 0$

Therefore: $\Phi^* = 2$ and we can determine RCL for a given α:

- $\alpha = 0.5$:
  $RCL = \{(x_1 = 1), (x_1 = 0), (x_2 = 1), (x_2 = 0), (x_3 = 0), (x_4 = 1)\}$
- $\alpha = 0.75$:
  $RCL = \{(x_1 = 1), (x_3 = 0)\}$

Note: Since we deal here with a maximization problem, the role of α is inverted w.r.t. the cost-based rule:
$\alpha = 1 \;\hat{=}\; \text{greedy}, \qquad \alpha = 0 \;\hat{=}\; \text{random}.$

Example: GRASP for (MAX-)SAT

- Selection from *RCL*:
  - If an unsatisfied clause contains only one non-instantiated variable, then try to satisfy this clause
  - Otherwise, select an element from *RCL* randomly
- Local Search:
  - 1-flip neighborhood structure
  - Best improvement step function

Example: Quadratic Assignment Problem (QAP)

Quadratic Assignment Problem (QAP)

Given: n locations indexed $\{1, \dots, n\}$, n facilities indexed $\{1, \dots, n\}$

$d_{i,j} \in \mathbb{R}$ for all $i, j = 1, \dots, n$: distance between location i and location j

$f_{i,j} \in \mathbb{R}$ for all $i, j = 1, \dots, n$: flow between facility i and facility j

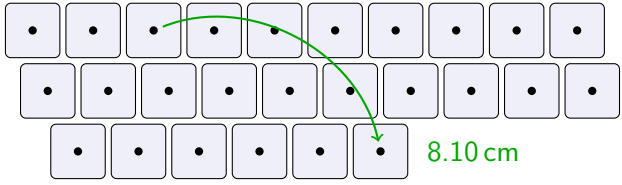
Goal: Assign each facility to its own (unique) location, i.e., find a permutation $\pi : \{1, \dots, n\} \to \{1, \dots, n\}$, where π_i represents the location assigned to the i-th facility, such that the following objective function is minimized:

$$c(\pi) = \sum_{i=1}^{n} \sum_{j=1}^{n} f_{i,j}\, d_{\pi_i,\pi_j} = \operatorname{tr}\big((f_{i,j})_{i,j=1}^{n}\, \Pi(\pi)\, ((d_{i,j})_{i,j=1}^{n})^{\top}\, \Pi(\pi)^{\top}\big)$$

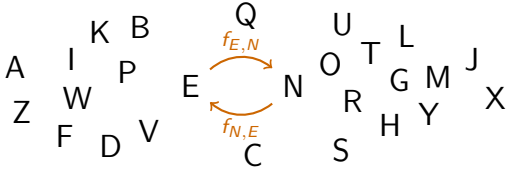
When seen as decision variable, $\Pi(\pi)$ contributes quadratically
$\Pi(\pi)$ is the permutation matrix associated with π

Application of QAP to keyboard layout

Locations:



Facilities:



Example: GRASP for QAP

Li et al. (1994)

- Greedy function: costs of interaction between new facility/location pairs and the already existing pairs in the partial solution
- *RCL* size is restricted through k
- Select facility/location pair randomly from *RCL*
- 2-exchange Local Search (i.e., neighborhood exchanging two facilities)

Extensions for GRASP

Select from *RCL* with Bias

- Components within *RCL* are selected with different probabilities, e.g., proportional to $1/c(e)$.

Automatic adjustment of α

- Set parameter randomly
- Reactive GRASP proposed by Prais and Ribeiro (2000) forcing self-adjusted restrictiveness for the *RCL*

procedure Reactive GRASP (minimization)

begin
  choose candidate values $\alpha_1, \ldots, \alpha_m$; $p_k \leftarrow 1/m$ for $k = 1, \ldots, m$; $x^* \leftarrow$ undefined;
  repeat:
    select index k with probability p_k;
    $x \leftarrow$ Randomized Greedy Heuristic (α_k); $x' \leftarrow$ Local Search (x);
    if x^* undefined **or** x' is better than x^* **then** $x^* \leftarrow x'$;
    $A_k \leftarrow$ average objective value of all solutions obtained with α_k;
    every N_α iterations **do** (δ steers the reweighting)
      $q_k \leftarrow \left(f(x^*)/A_k\right)^{\delta}$ and $p_k \leftarrow q_k / \sum_{l=1}^{m} q_l$ for $k = 1, \ldots, m$;
  until stopping criteria satisfied; **return** x^*;
end

Extensions for GRASP (continued)

Usage of a memory, e.g., see Resende and Ribeiro (2005)

- Use an elite of good solutions to control the construction process.

Simple parallelization

- We can run GRASP independently on several cores.

Conclusions for GRASP

- Simple to implement extension of construction heuristics and Local Search
- Few parameters
- Many applications
- Often relatively fast while producing relatively good solutions

Variable Neighborhood Search (VNS)

Variable Neighborhood Search (VNS)

Mladenović and Hansen (1997)

Method takes advantage of the following observations:

1. A local optimum with respect to one neighborhood structure is **not** necessarily a local optimum with respect to another.
2. A global optimum is a local optimum with respect to all possible neighborhood structures.
3. For many problems good local optima are relatively close together.

Neighborhood structures $\mathcal{N}_1, \ldots, \mathcal{N}_{l_{\max}}$ are changed systematically.

- Use **small** neighborhood structures to quickly find a local optimum
- Use **large** neighborhood structures to explore new regions of the search space

Variable Neighborhood Descent (VND)

procedure Variable Neighborhood Descent (x)

begin
  $l \leftarrow 1$; // index of neighborhood structure
  repeat:
    find an $x' \in \mathcal{N}_l(x)$ with $f(x') \leq f(x'')$, $\forall x'' \in \mathcal{N}_l(x)$;
    if $f(x') < f(x)$ **then**
      $x \leftarrow x'$;
      $l \leftarrow 1$;
    else
      $l \leftarrow l + 1$;
  until $l > l_{\max}$;
  return x;
end

Variable Neighborhood Descent (VND)

Step function: usually best or first improvement

A solution generated by VND is a local optimum with respect to **all** neighborhood structures $\mathcal{N}_1, \ldots, \mathcal{N}_{l_{\max}}$.

Order of neighborhood structures:

- mostly, according to the size (from small to large) or complexity of the evaluation
- or randomly

Example: Vehicle Routing Problem (VRP)

- $\mathcal{N}_1(x)$: 2-exchange within a single tour
- $\mathcal{N}_2(x)$: Move city from one tour to another
- $\mathcal{N}_3(x)$: Exchange two cities between two tours
- $\mathcal{N}_4(x)$: 3-exchange within a single tour
- ...

About the Neighborhood Structures $N_1, \ldots, N_{k_{\max}}$

At least theoretically, to avoid getting stuck in a local optimum, the union of all $N_1(x), \ldots, N_{k_{\max}}(x)$ should cover the feasible search space S:

$$S \subseteq \{x\} \cup N_1(x) \cup N_2(x) \cup \cdots \cup N_{k_{\max}}(x), \ \forall x \in S$$

The individual neighborhoods can but need not be disjoint.

Often nested neighborhoods are a more efficient choice:

$$N_1(x) \subseteq N_2(x) \subseteq \cdots \subseteq N_{k_{\max}}(x), \forall x \in S$$

This can be achieved, e.g., by implementing neighborhood N_k as up to k consecutive moves in neighborhood N_1.

Reduced Variable Neighborhood Search (RVNS)

RVNS is a local-search-free VNS variant, useful for very large instances where a local search would be too costly.

Basic idea: Select neighbors randomly (random neighbor step function) from systematically increasing neighborhoods.

Again, there is a pool of different neighborhood structures $N_1, \ldots, N_{k_{\max}}$, but with the following **properties**:

- These neighborhoods are often significantly larger and cannot be fully searched in general.
- The application order typically follows
  - increasing complexity, or
  - the "structural distance between the new solutions and the initial solution".

procedure Reduced VNS (x)

begin
  $x \leftarrow$ initial solution;
  repeat:
    $k \leftarrow 1$;
    repeat:
      shaking: generate a random $x' \in N_k(x)$;
      if $f(x') < f(x)$ **then**
        $x \leftarrow x'$;
        $k \leftarrow 1$;
      else
        $k \leftarrow k + 1$;
    until $k > k_{\max}$;
  until stopping criteria satisfied;
  return x;
end

Basic Variable Neighborhood Search (BVNS)

- RVNS is extended by a simple local search on a more restricted neighborhood structure.
- Combination of deterministic and stochastic moves in the search space.
- Often successive neighborhoods are nested.

procedure Basic VNS (x)

begin
  $x \leftarrow$ initial solution;
  $x \leftarrow$ Local Search(x);
  repeat:
    $k \leftarrow 1$;
    repeat:
      shaking: generate a random $x' \in N_k(x)$;
      $x' \leftarrow$ Local Search(x');
      if $f(x') < f(x)$ **then**
        $x \leftarrow x'$;
        $k \leftarrow 1$;
      else
        $k \leftarrow k + 1$;
    until $k > k_{\max}$;
  until stopping criteria satisfied;
  return x;
end

General Variable Neighborhood Search (GVNS)

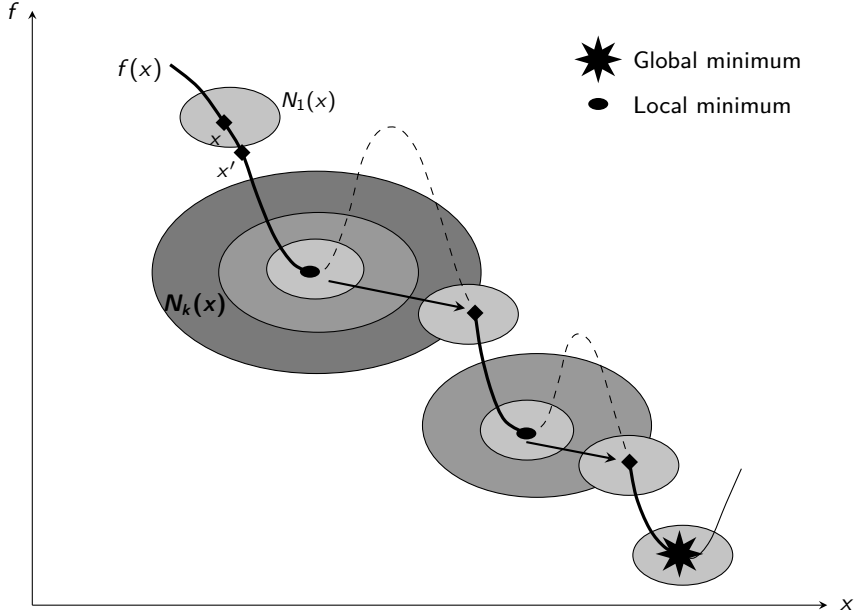
- Combines BVNS with VND as local search method.
- Here, moves in $N_1, \ldots, N_{k_{\max}}$ are also called shaking moves.
- The neighborhood structures of the shaking part of BVNS ($N_1, \ldots, N_{k_{\max}}$) and of VND ($\mathcal{N}_1, \ldots, \mathcal{N}_{l_{\max}}$) are **in general not identical**!
- **Question:** Why is shaking usually more effective than restarting from an independent random solution?
  - For many problems local optima are relatively close together.
  - Fewer iterations needed to reach a local optimum.

procedure General VNS (x) [short version]

begin
  $x \leftarrow$ initial solution;
  $x \leftarrow$ VND(x) with neighborhood structures $\mathcal{N}_1, \ldots, \mathcal{N}_{l_{\max}}$;
  repeat:
    $k \leftarrow 1$;
    repeat:
      shaking: generate a random $x' \in N_k(x)$;
      $x' \leftarrow$ VND(x') with neighborhood structures $\mathcal{N}_1, \ldots, \mathcal{N}_{l_{\max}}$;
      if $f(x') < f(x)$ **then**
        $x \leftarrow x'$;
        $k \leftarrow 1$;
      else
        $k \leftarrow k + 1$;
    until $k > k_{\max}$;
  until stopping criteria satisfied;
  return x;
end

procedure General VNS (x) [long version]

begin
  $x \leftarrow$ initial solution;
  $x \leftarrow$ VND(x) with neighborhood structures $\mathcal{N}_1, \ldots, \mathcal{N}_{l_{\max}}$;
  repeat:
    $k \leftarrow 1$;
    repeat:
      shaking: generate a random $x' \in N_k(x)$;
      $l \leftarrow 1$;
      repeat:
        find an x'' with $f(x'') \leq f(x'''),\ \forall x''' \in \mathcal{N}_l(x')$;
        if $f(x'') < f(x')$ **then**
          $x' \leftarrow x''$; $l \leftarrow 1$;
        else
          $l \leftarrow l + 1$;
      until $l > l_{\max}$;
      if $f(x') < f(x)$ **then**
        $x \leftarrow x'$;
        $k \leftarrow 1$;
      else
        $k \leftarrow k + 1$;
    until $k > k_{\max}$;
  until stopping criteria satisfied;
  return x;
end



General Variable Neighborhood Search, adapted from Hansen and Mladenović (2009)

Variable Neighborhood Decomposition Search (VNDS)

VNS variants have been frequently applied to obtain approximate solutions of many combinatorial optimization problems

For very large instances the required computation time might still be too high.

- **Common remedy**: Decomposition of the problem

VNDS: Variation or extension of (basic) VNS for solving large instances

procedure VNDS (x)

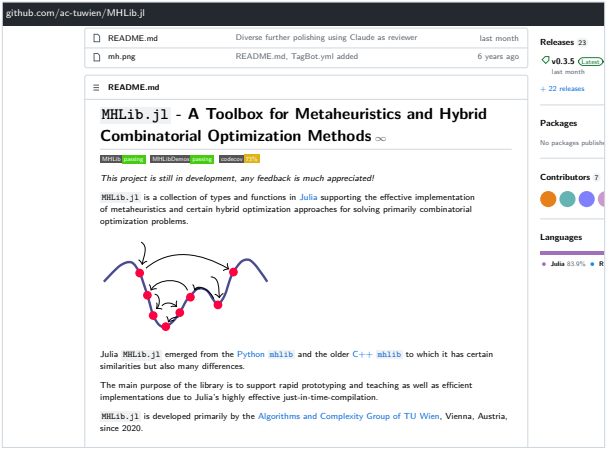
begin
  $x \leftarrow$ initial solution;
  repeat:
    $k \leftarrow 1$;
    repeat:
      shaking: generate a random $x' \in N_k(x)$;
      $Y \leftarrow$ set of solution attributes w.r.t. which x differs from x';
      $x' \leftarrow$ Local Search (x') or Improvement Heuristic (x'),
          where optimization carried out just over attributes in Y;
      if $f(x') < f(x)$ **then**
        $x \leftarrow x'$; $k \leftarrow 1$;
      else
        $k \leftarrow k + 1$;
    until $k > k_{\max}$;
  until stopping criteria satisfied;
  return x;
end

Conclusions for VNS

- Highly versatile metaheuristic with many applications
- Many variants and extensions relying on the basic principles of VNS
- Careful design necessary:
  - Choice of the right framework w.r.t. the problem at hand
  - Selection of neighborhood structures
  - Ordering and relations between neighborhood structures
  - Applied improvement heuristic
  - Balance between deterministic and stochastic elements
- Further variants:
  Adaptive VNS, Parallel VNS, Primal-Dual VNS, ...

Brief live-demo of metaheuristics using MHLib (Julia package)

- The package provides ready-to-use data structures and core routines of established metaheuristics (some of them already encountered, others to be discussed in future lectures)
- URL of package: `https://github.com/ac-tuwien/MHLib.jl`
- Further insights during the following live demo ...



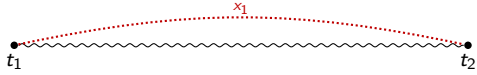
Variable Depth Search

Variable Depth Search (VDS)

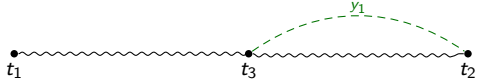
- Complex moves are built as concatenations of simple moves.
  - E.g., in MAX-SAT a 3-flip move can be seen as a concatenation of three 1-flip moves.
- Number of simple moves is variable.
- Moves are not independent of each other.
- Neighborhood is limited by restrictions.
- Promising move combinations will be followed,
  less reasonable move combinations are discarded.

Example: Lin-Kernighan Heuristic (LK) for Symmetric TSP

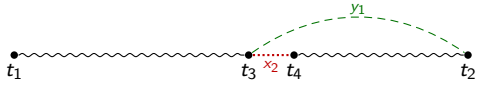
- **Remove** $x_1 = t_1t_2$ from the tour: a Hamiltonian path from t_1 to t_2 remains



- **Add** $y_1 = t_2t_3$: a cycle closes at the end of the path, giving a δ-path

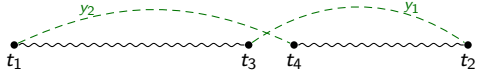


- **Remove** $x_2 = t_3t_4$, the other cycle edge at t_3: again a Hamiltonian path, now from t_1 to t_4



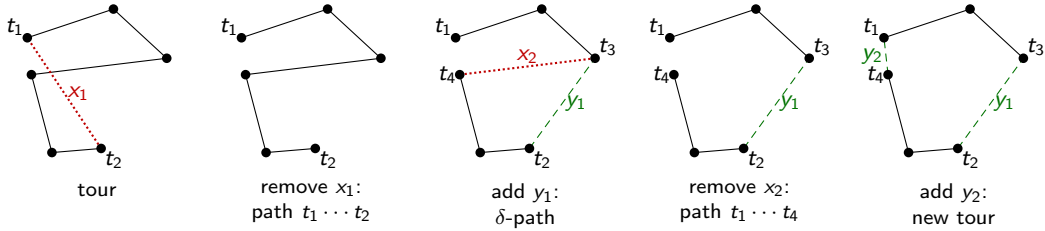
increasing the depth of search

- **Add** $y_2 = t_4t_1$: a tour again (here a 2-opt move) — **or continue** with $y_2 = t_4t_5$ to the next δ-path



Lin and Kernighan (1973); the roles of t_2, t_3, t_4 for t_1 (in level 1) are taken by $t_{2i}, t_{2i+1}, t_{2i+2}$ (in level i)

Example: Lin-Kernighan Heuristic (LK) for Symmetric TSP



- Complex moves are constructed from sequences of dependent edge-exchange moves
- Variable number of edges can be exchanged.
- Neighborhood is exponentially large, but one LK move (greedy, limited backtracking) is found in polynomial time.
- For each edge-exchange move a δ-path is considered as an intermediate result, which is created by adding an edge to the Hamiltonian path (see above).

Allowed Edge-exchange Moves in LK

LK considers a restricted set of promising k-opt moves with variable k by modifying two sets X and Y of edges to be removed and added, respectively (Helsgaun, 2000).

- **Sequential Exchange Criterion**
  Path $x_1, y_1, \ldots, x_i, y_i, x_{i+1}, y_{i+1}, \ldots, x_k, y_k$ is X-Y-alternating, i.e.,

$$x_j \in X \quad \text{and} \quad y_j \in Y \qquad \forall j \in \{1, \ldots, k\}$$

- **Disjunctivity Criterion**: X and Y are disjoint ($\rightsquigarrow$ sequence must be repetition-free)
- **Feasibility Criterion**: For $i \geq 3$, the edge $x_i = (t_{2i-1}, t_{2i})$ must be chosen such that joining t_{2i} to the first vertex t_1 yields a Hamiltonian cycle (avoids subtours).
  $\rightarrow$ restricts the search to k-opt moves realizable by a 2- or 3-opt move followed by a sequence of 2-opt moves
- **Positive Gain Criterion**: $G_i := \sum_{j=1}^{i} \left(c(x_j) - c(y_j)\right) > 0, \quad \forall i \in \{1, \ldots, k\}$

Remarks for VDS

- The Lin-Kernighan Heuristic for the TSP is a prominent example of which there are many variations.
  - E.g., depth restriction
- The principle of Variable Depth Search is often used in a simplified way to reasonably restrict large neighborhoods:
  - (Partial-)moves are only applied if they appear promising according to a certain criterion.
- Example: MAX-SAT:
  Instead of evaluating all k-flip moves just examine those variables that occur in unsatisfied clauses.
- Often, complex neighborhoods are specified in the following way:
  try to identify particularly "expensive" (i.e., inefficient) solution components and then try to find a "favorable" replacement.

Bibliography I

T. A. Feo and M. G. Resende. A probabilistic heuristic for a computationally difficult set covering problem. *Operations Research Letters*, 8(2):67–71, 1989.

T. A. Feo and M. G. Resende. Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6:109–133, 1995.

P. Hansen and N. Mladenović. Variable neighborhood search methods. In C. A. Floudas and P. M. Pardalos, editors, *Encyclopedia of Optimization*, pages 3975–3989. Springer, 2nd edition, 2009.

K. Helsgaun. An effective implementation of the Lin–Kernighan traveling salesman heuristic. *European Journal of Operational Research*, 126(1):106–130, 2000.

Y. Li, P. M. Pardalos, and M. G. Resende. A greedy randomized adaptive search procedure for the quadratic assignment problem. In P. M. Pardalos and H. Wolkowicz, editors, *Quadratic Assignment and Related Problems*, volume 16 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, pages 237–261. American Mathematical Society, 1994.

S. Lin and B. W. Kernighan. An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21(2):498–516, 1973.

H. R. Lourenço, O. C. Martin, and T. Stützle. Iterated local search. In F. Glover and G. A. Kochenberger, editors, *Handbook of Metaheuristics*, pages 320–353. Springer, 2003.

N. Mladenović and P. Hansen. Variable neighborhood search. *Computers & Operations Research*, 24(11): 1097–1100, 1997.

M. Prais and C. C. Ribeiro. Reactive GRASP: An application to a matrix decomposition problem in TDMA traffic assignment. *INFORMS Journal on Computing*, 12(3):164–176, 2000.

Bibliography II

M. G. Resende and C. C. Ribeiro. GRASP with path-relinking: Recent advances and applications. In T. Ibaraki, K. Nonobe, and M. Yagiura, editors, *Metaheuristics: Progress as Real Problem Solvers*, volume 32 of *Operations Research/Computer Science Interfaces Series*, pages 29–63. Springer, 2005.